TOPIC 6.1

Polynomial Regression

Fitting curves with the tools of linear regression

WHAT YOU WILL LEARN

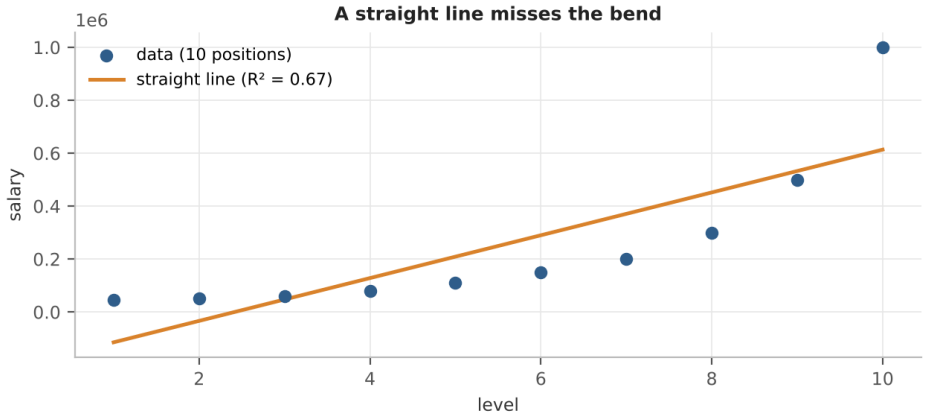
- Explain **why** a straight line fails on curved data
- Write a **polynomial of degree n** and say what the degree controls
- Understand the key trick: **add power columns, then run ordinary linear regression**
- Use `PolynomialFeatures` and `LinearRegression` in scikit-learn, including how to predict
- Choose the degree sensibly and avoid **overfitting** and wild **extrapolation**

Builds on the linear regression and ML foundation notes (5.1 to 5.6). Linear regression fits a straight line. Here we keep exactly the same method, but give it extra input columns so that the fitted line can bend. The risks of a model that is too flexible were covered in notes 5.3 (overfitting), 5.4 (bias and variance) and 5.5 (regularization); they matter a great deal here.

1. The problem: data that bends

Think of salaries in a company. A junior engineer earns a little more than a trainee, but a CEO earns vastly more than a vice president. Salary does not rise in equal steps with seniority; it rises **faster and faster**. Here are the ten levels used in class:

Level	1	2	3	4	5	6	7	8	9	10
Salary (thousands)	45	50	60	80	110	150	200	300	500	1,000



Straight-line regression on the salary data. The line is too high for the middle levels and too low at both ends.

The best straight line explains only about **67%** of the variation ($R^2 = 0.67$). Worse, it predicts a salary of about **330,379** for level 6.5, although the real salaries of levels 6 and 7 are 150,000 and 200,000. This is **underfitting**: the model is too simple for the shape of the data (note 5.3). No amount of extra training fixes it, because the line itself is the wrong shape.

2. The idea: let the equation have powers of x

A straight line is the equation $y = b_0 + b_1x$. Polynomial regression adds higher **powers** of the same input:

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \ \ldots \ + b_nx^n$$

The **degree** n is the highest power used. It controls how many bends the curve can have.

Degree	Equation	Shape
1	$y = b_0 + b_1x$	Straight line
2	$y = b_0 + b_1x + b_2x^2$	One bend (U or upside-down U)
3	$\ldots + b_3x^3$	Up to two bends (an S shape)
n	$\ldots + b_nx^n$	Up to n − 1 bends

> **DEFINITION** **Polynomial regression**
> A regression model whose output is a sum of coefficients times powers of the input (x, x^2, x^3 …). The coefficients b are learned from the data, exactly as in linear regression.

3. The key trick: new columns, same linear regression

The equation looks non-linear because x is squared. But look at it from the model's point of view. Suppose we make **new columns** out of x:

x	x^2	x^3
1	1	1
2	4	8
3	9	27

(The column x is the original input for the values 1, 2 and 3; x^2 and x^3 are the new, computed columns.) If we call these columns x_1, x_2 and x_3, the equation becomes $y = b_0 + b_1x_1 + b_2x_2 + b_3x_3$. That is an ordinary **multiple linear regression**, which we already know how to solve. The word *linear* refers to the **coefficients**, which are only multiplied and added, not to the shape of the curve.

> **KEY IDEA**
> - Polynomial regression = **make power columns**, then run **linear regression** on them
> - The curve can bend, yet the model is still found with the same least-squares method
> - This is why one tool, `LinearRegression`, serves both

With several inputs, say x_1 and x_2, a degree-2 expansion produces [x_1, x_2, x_1^2, x_1x_2, x_2^2]. The product x_1x_2 is an **interaction** term: it lets the effect of one feature depend on the other. (The class slides show only the squares for simplicity.)

4. Doing it in scikit-learn

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

data = pd.read_csv("emp_sal.csv")
X = data.iloc[:, 1:2].values      # Level  (kept 2-D: a column)
y = data.iloc[:, 2].values        # Salary

poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)    # columns: 1, x, x^2, x^3, x^4

model = LinearRegression().fit(X_poly, y)

# To predict, transform the new value the SAME way first
model.predict(poly.transform([[6.5]]))
```

Four details beginners often miss:

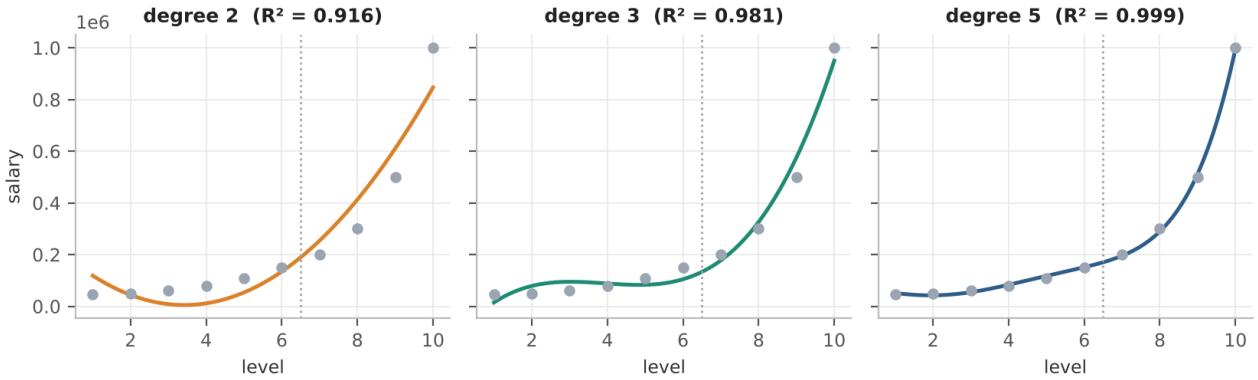
- `PolynomialFeatures(degree=d)` only **creates the columns**. The model that learns is still `LinearRegression`.
- Its first output column is all ones (the bias). `LinearRegression` already has an intercept, so this is harmless; use `include_bias=False` to drop it.
- The input must be **2-D** (`X` with shape rows × 1), which is why the code uses `1:2` rather than `1`.
- For new data call `poly.transform(...)`, not `fit_transform`, and never feed a raw value straight into the model.

The class code also compares this with a support vector regressor using a polynomial kernel. That is a different algorithm that reaches a similar curved fit by another route; the idea of the notes above stays the same.

5. Results on the salary data

We fitted degrees 1 to 5 to all ten points and predicted level 6.5:

Degree	R^2 on the 10 points	Predicted salary at level 6.5
1	0.6690	330,379
2	0.9162	189,498
3	0.9812	133,259
4	0.9974	158,862
5	0.9994	170,446



Degrees 2, 3 and 5 on the salary data. The dotted line marks level 6.5.

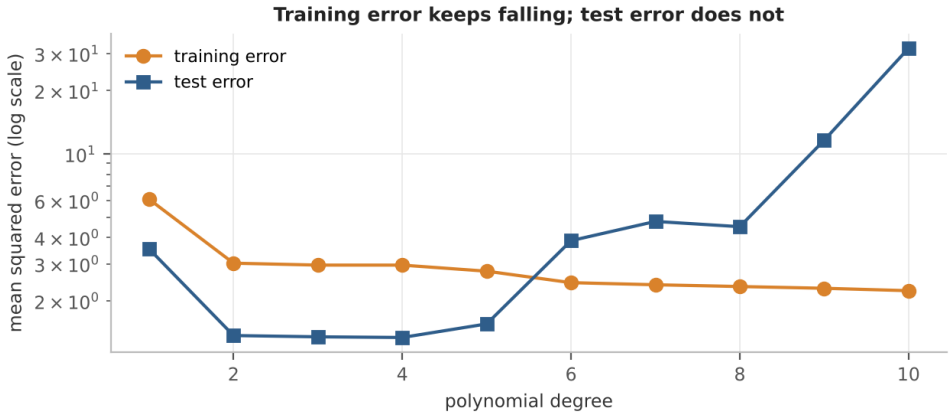
R^2 rises with every extra degree, and the straight line is far off. The curved models land in a much more sensible range (133,259 to 189,498), but they **do not agree** with each other, because with only ten points the data cannot pin down the exact curve.

> **NOTE** **Reading the class example honestly**
>
> In class an employee claims a previous salary of 161K at level 6.5, and the model's prediction is used to judge whether he is bluffing. Degree 4 gives about 158,862 and degree 3 about 133,259, so the verdict depends on which degree you pick. Treat the result as a rough guide, not as proof.

6. Choosing the degree

A higher degree can always fit the training points better, because it has more freedom. But a curve that wiggles through every point is also fitting the noise, so it does badly on new data. This is **overfitting** (note 5.3), and it is the bias–variance trade-off of note 5.4 in action: low degree means high bias, high degree means high variance.

To see this we generated 30 noisy points from a smooth curved relationship, trained on 60% and tested on the remaining 40%:



Training error falls steadily as the degree grows, but the test error is lowest at a small degree and then worsens.

Degree	1	2	3	5	10
Training error	6.08	3.02	2.96	2.77	2.24
Test error	3.53	1.37	1.35	1.56	31.70

Degree 1 is far too simple (test error 3.5). Degree 2 matches the truth, and degree 10 has a lower training error (2.24) than degree 2 (3.02) yet a higher test error (31.70 against 1.37). Five-fold cross-validation on all 30 points prefers degree 2.

> **KEY IDEA** **A practical recipe**
>
> - Start with degree 2 or 3; try a few more values, never a huge one
> - Judge each degree on **held-out data** (a test set or cross-validation), not on training R^2
> - Pick the **smallest** degree that works well
> - If the curve is still too wiggly, add regularization (Ridge or Lasso, note 5.5) instead of a lower degree

The class data set has only ten rows, so the class code uses all of it for fitting and does not split. That is fine for learning the idea, but with real data you would validate as described above.

7. Many features: scaling and column growth

Power columns grow quickly with the number of inputs and the degree. This table counts the columns `PolynomialFeatures` creates (without the bias column):

Original features	Degree 2	Degree 3	Degree 5
1	2	3	5
2	5	9	20
5	20	55	251
10	65	285	3002

Ten features at degree 3 already give 285 columns, which makes overfitting easy with a small data set. Powers also create very different magnitudes ($x = 10$ gives $x^5 = 100{,}000$), which is exactly the situation scaling (note 5.2) solves. A robust pattern is therefore:

```
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import Ridge

model = make_pipeline(PolynomialFeatures(degree=3, include_bias=False),
                      StandardScaler(),
                      Ridge(alpha=1.0))
model.fit(X_train, y_train)      # scaling is learned from train only
```

8. Danger: predicting outside the data

A polynomial is only trustworthy **inside the range of the data it was trained on** (here levels 1 to 10). Outside that range it can swing wildly. For a level-12 position (two levels above the data) the degree-2 model predicts about 1,435,894 and the degree-5 model about 3,592,972, a gap of more than two million between two models that agree reasonably well inside the data. Use polynomial models for **interpolation** (between known points), not for forecasting far beyond them.

Common mistakes

COMMON MISTAKE

- Calling a polynomial model "non-linear regression" in the sense of a different algorithm: it is linear regression on power columns
- Choosing the degree because training R^2 is highest (it always rises)
- Forgetting to transform the new input with the same `PolynomialFeatures` before predicting
- Passing a 1-D array instead of a 2-D column
- Extrapolating far outside the training range
- Using a very high degree without scaling or regularization

Summary

SUMMARY

- Polynomial regression fits **curves** by adding power columns (x^2, x^3 ...) and using ordinary linear regression
- The **degree** sets the flexibility: too low underfits, too high overfits
- In scikit-learn: `PolynomialFeatures` to build columns, then `LinearRegression` (or Ridge) to fit; call `transform` on new data
- Pick the degree with validation data and prefer the smallest that works
- Scale features and consider regularization when degree or feature count grows
- Stay inside the range of the training data when predicting

Practice questions

Try each question before reading the answer underneath it.

Q1. Why is $y = b_0 + b_1x + b_2x^2$ still called a linear model?

Answer. It is linear in the coefficients b_0, b_1, b_2: they are only multiplied by columns and added. Once x^2 is treated as a column, it is ordinary multiple linear regression.

Q2. What does degree 3 allow that degree 2 does not?

Answer. Up to two bends instead of one, so an S-shaped curve is possible.

Q3. A degree-2 expansion of the features $[x_1, x_2]$ gives which columns?

Answer. x_1, x_2, x_1^2, x_1x_2, x_2^2 (plus an optional column of ones).

Q4. Why must you call `poly.transform` before `model.predict`?

Answer. The model was trained on the expanded columns, so a new value must be expanded in exactly the same way.

Q5. The training R^2 goes from 0.99 to 0.9995 when you raise the degree. Is the model better?

Answer. Not necessarily. Training fit always improves with degree. Check on held-out data before deciding.

Q6. Why is predicting level 15 with a degree-5 model risky?

Answer. Level 15 is outside the training range (1 to 10), where the polynomial can swing wildly with no data to support it.